

1. (A) Below is the implementation of a subroutine that for each node calculates the number of edges leading away from it (“out links”).

```
1  FILENAME = 'data/web-Google-10k.txt'
2
3  with open(FILENAME) as file:
4      lines = [line.strip().split('\t') for line in file]
5      edges = [[int(src), int(dst)] for (src, dst) in lines[4:]]
6      nodes = set([node for edge in edges for node in edge])
7
8      links = { node: [] for node in nodes }
9      for (src, dst) in edges:
10         links[src].append(dst)
11
12     for node in nodes:
13         print(node, len(links[node]))
```

Listing 1. out-links.py

- (B) Below is the implementation of a subroutine that fetches all the dead-end nodes in the graph but does not report the IDs of nodes that are dead-end nodes. There are in total 1235 in the graph.

```
1  FILENAME = 'data/web-Google-10k.txt'
2
3  with open(FILENAME) as file:
4      lines = [line.strip().split('\t') for line in file]
5      edges = [[int(src), int(dst)] for (src, dst) in lines[4:]]
6      nodes = set([node for edge in edges for node in edge])
7
8      links = { node: [] for node in nodes }
9      for (src, dst) in edges:
10         links[src].append(dst)
11
12     dends = sum(1 for node in nodes if len(links[node]) == 0)
13     print(dends)
```

Listing 2. dead-ends.py

(C) From the implementation below, the node with original ID **486980** has the highest PageRank score.

```

1  import numpy as np
2
3  FILENAME = 'data/web-Google-10k.txt'
4  EPSILON = 0.0001
5  BETA = 0.85
6
7  def build_trans_matrix(init_matrix: np.ndarray, beta: float = BETA) -> np.ndarray:
8      num_nodes = init_matrix.shape[0]
9      const_vec = np.repeat(1 / num_nodes, num_nodes)
10     d_entries = init_matrix.sum(axis = 0)
11     dead_ends = np.where(d_entries == 0)[0]
12
13     for dead_end in dead_ends:
14         init_matrix[:, dead_end] = 1 / num_nodes
15         d_entries[dead_end] = 1
16
17     init_matrix /= d_entries
18     return beta * init_matrix + (1 - beta) * const_vec
19
20 def power_iteration(trans_matrix: np.ndarray, epsilon: float = EPSILON) -> np.ndarray:
21     n_size = trans_matrix.shape[0]
22     r_prev = np.repeat(1 / n_size, n_size)
23     r_next = trans_matrix @ r_prev
24
25     while np.linalg.norm(r_next - r_prev, ord=1) >= epsilon:
26         r_prev = r_next
27         r_next = trans_matrix @ r_prev
28
29     return r_next
30
31 with open(FILENAME) as file:
32     lines = [line.strip().split('\t') for line in file]
33     edges = [(int(src), int(dst)) for (src, dst) in lines[4:]]
34     nodes = set([node for edge in edges for node in edge])
35
36     node_index = {node: index for index, node in enumerate(nodes)}
37     index_node = {index: node for node, index in node_index.items()}
38
39     n_size = len(nodes)
40     matrix = np.zeros((n_size, n_size))
41     for (src, dst) in edges:
42         src = node_index[src]
43         dst = node_index[dst]
44         matrix[dst][src] = 1
45
46     M_matrix = build_trans_matrix(matrix)
47     r_vector = power_iteration(M_matrix)
48     max_node = index_node[r_vector.argmax()]
49     n_degree = sum(1 for _, dst in edges if dst == max_node)
50     print(max_node, n_degree)

```

Listing 3. pagerank-adj-mat.py

(D) From **Listing 3**, a total of **155** nodes point to the node with highest PageRank. This is reasonable because it is the same output obtained from the PageRank subroutine from [NetworkX](#) package.

2. Below are implementations of some of the ways for speeding-up PageRank.

1. Representing Transition Matrices.

```

1  import numpy as np
2
3  FILENAME = 'data/web-Google.txt'
4  EPSILON = 0.0001
5  BETA = 0.85
6
7  def update_ranks(matrix: dict, r_prev: np.ndarray) -> np.ndarray:
8      r_size = r_prev.shape[0]
9      r_next = np.repeat((1 - BETA) / r_size, r_size)
10     for src, dst_list in matrix.items():
11         r_next[dst_list] += BETA * r_prev[src] / len(dst_list)
12     return r_next
13
14     with open(FILENAME) as file:
15         lines = [line.strip().split('\t') for line in file]
16         edges = [(int(src), int(dst)) for (src, dst) in lines[4:]]
17         nodes = set([node for edge in edges for node in edge])
18
19         node_index = {node: index for index, node in enumerate(nodes)}
20         index_node = {index: node for node, index in node_index.items()}
21
22         matrix = {}
23         for (src, dst) in edges:
24             src = node_index[src]
25             dst = node_index[dst]
26             matrix[src] = matrix.get(src, []) + [dst]
27
28         n_size = len(nodes)
29         r_prev = np.repeat(1 / n_size, n_size)
30         r_next = np.zeros(n_size)
31
32         while np.linalg.norm(r_next - r_prev, ord=1) >= EPSILON:
33             r_prev = r_next
34             r_next = update_ranks(matrix, r_prev)
35
36         max_node = index_node[r_next.argmax()]
37         n_degree = sum(1 for _, dst in edges if dst == max_node)
38         print(max_node, n_degree)

```

Listing 4. pagerank-adj-lst.py

Running **Listing 4** on the entire `web-Google.txt` dataset took 100 seconds. In contrast, **Listing 3** failed to run due to the sheer size of the adjacency matrix totaling 5 TB of disk space. This matrix representation is a performance improvement as the space required is roughly linear in the number of non-zero entries rather than quadratic in the size of the matrix.¹

2. Distributed Map Reduce on n Stripes.

Below is **Listing 5**, a pseudo-implementation of PageRank with MapReduce. It took 200 seconds to process the entire `web-Google.txt` dataset. This drop in performance is attributed to its sequential nature. In it, we observe a lack of concurrency where the map-reduce function uses `starmap` and `reduce` in a serial manner, without leveraging parallelism or concurrency. In theory, since map-reduce is inherently parallelizable, running the code on multiple cores or nodes in a distributed cluster would significantly speed up computation and offer better performance.

¹Solution outline weakly inspired from: <http://infolab.stanford.edu/~ullman/mmds/ch5.pdf>.

```

1 import numpy as np
2
3 from typing import Callable
4 from functools import reduce
5 from itertools import starmap
6
7 FILENAME = 'data/web-Google.txt'
8 EPSILON = 0.0001
9 BETA = 0.85
10
11 def generate_vector_stripes(vector: np.ndarray, stripe_size: int) → np.ndarray:
12     n_value = vector.shape[0]
13     indices = np.arange(stripe_size, n_value, stripe_size)
14     stripes = np.hsplit(vector, indices)
15     return stripes
16
17 def mapper(matrix: dict, r_prev: np.ndarray) → np.ndarray:
18     r_size = r_prev.shape[0]
19     r_next = np.repeat((1 - BETA) / n_size, n_size)
20     for src, dst_list in matrix.items():
21         for dst in dst_list:
22             r_next[dst] += BETA * r_prev[src % r_size] / len(dst_list)
23     return r_next
24
25 def reducer(accumulator: np.ndarray, vector: np.ndarray) → np.ndarray:
26     return accumulator + vector
27
28 def map_reduce(mapper: Callable, reducer: Callable, args: tuple) → np.ndarray:
29     mapped = starmap(mapper, args)
30     return reduce(reducer, mapped)
31
32 with open(FILENAME, 'r') as file:
33     lines = [line.strip().split('\t') for line in file]
34     edges = [[int(src), int(dst)] for (src, dst) in lines[4:]]
35     nodes = set([node for edge in edges for node in edge])
36
37     node_index = {node: index for index, node in enumerate(nodes)}
38     index_node = {index: node for node, index in node_index.items()}
39
40     n_size = len(nodes)
41     vector = np.zeros(n_size)
42
43     stripe_size = 30197
44     stripe_nums = n_size // stripe_size
45     matrix_stripes = [{ } for _ in range(stripe_nums)]
46     vector_stripes = generate_vector_stripes(vector, stripe_size)
47
48     for (src, dst) in edges:
49         src = node_index[src]
50         dst = node_index[dst]
51         matrix_stripe = matrix_stripes[src // stripe_size]
52         matrix_stripe[src] = matrix_stripe.get(src, []) + [dst]
53
54     stripes = zip(matrix_stripes, vector_stripes)
55     reduced = map_reduce(mapper, reducer, stripes)
56
57     while np.linalg.norm(reduced - vector, ord=1) ≥ EPSILON:
58         vector = reduced
59         vector_stripes = generate_vector_stripes(vector, stripe_size)
60         stripes = zip(matrix_stripes, vector_stripes)
61         reduced = map_reduce(mapper, reducer, stripes)
62
63     max_node = index_node[reduced.argmax()]
64     n_degree = sum(1 for _, dst in edges if dst == max_node)
65     print(max_node, n_degree)

```

Listing 5. pagerank-map-red.py